RTSE-KT

Project Overview

This project is a rooftop solar engineering tool that helps with layout design and computation. The tool assists in planning and optimizing solar panel layouts on rooftops.

Project Structure

```
RTSE-KT/
├── _pycache_/        # Python bytecode cache directory
├── .idea/            # IntelliJ/PyCharm IDE configuration
├── outputs/          # Generated output files directory
├── rtse_venv/        # Python virtual environment
├── .env              # Environment variables configuration
├── call_graph.*      # Call graph visualization files
├── Python source files
└── Configuration files
|_ report.sql  #report sql for reporting the new data insertions
```

Key Files and Their Purposes

Core Engine Files

- `full_roof_layout_engine.py` : Main engine for roof layout calculations
- `layout_funcs.py` : Helper functions for layout operations
- `layoutEngine2.py` : Secondary/alternative layout engine (newer version)
- `compute_htn.py` : handles HTN (Hierarchical Task Network) computations

Data Management

- `dataloader.py` : Handles data input/output operations
- `params.py` : Configuration parameters and constants

Runner and Testing

- `mainrunner.py` : Main entry point of the application
- `testing.ini` : Test configuration file

Configuration and Requirements

- `requirements.txt` : Python package dependencies
- `qodana.yaml` : Code quality analysis configuration
- `.env` : Environment variables and configuration

Documentation and Output

- `call_graph.pdf` : Visual representation of code dependencies
- `call_graph.py` : Script to generate code dependency graph
- `report.html` : Generated reports
- `RTSE-KT1.png` : Project profiling diagram or screenshot
- `Python Call Graph.html` : Interactive code dependency visualization

Typical Workflow

1. **Data Input**
   - Configuration loaded from `params.py`
   - Environment variables from `.env`
   - Data loaded through `dataloader.py`
2. **Processing**
   - `mainrunner.py` initiates the process
   - Layout calculations performed by `full_roof_layout_engine.py`
   - Supported by various layout functions from `layout_funcs.py`
   - Additional computations handled by `compute_htn.py`
3. **Output Generation**
   - Results saved to `outputs/` directory
   - Reports generated in HTML format

Development Environment

The project uses:

- Python (with a dedicated virtual environment in `rtse_venv/`)
- Python version 3.9.13
- PyCharm/IntelliJ IDE (`.idea/` directory)
- Code quality tools (Qodana configuration)
- Automated documentation generation

Suggested Knowledge Transfer Steps

1. **Environment Setup**
   - Clone repository
   - Create virtual environment
   - Install dependencies from `requirements.txt`
   - Configure `.env` file
2. **Code Understanding**
   - Start with `mainrunner.py`
   - Review `params.py` for configuration
   - Study the layout engine implementation
   - Understand data flow through `dataloader.py`
3. **Testing**
   - Review `testing.ini` configuration
   - Run test cases
   - Verify output generation
4. **Documentation**
   - Review generated call graphs
   - Study HTML reports
   - Understand configuration parameters

Common Tasks

1. **Running the Application**

```
python mainrunner.py
```

2. **Generating Documentation**

```
python call_graph.py
```

Notes for Knowledge Transfer

- Review any external dependencies and their purposes
- Study the layout algorithms implemented in the engines
- Understand the configuration parameters and their impacts
- Review output formats and interpretation

Adding a New Location to the System

1. Entry Point Configuration

The main entry point is `mainrunner.py`. This file needs to be updated with new location information.

2. Location Configuration Steps

2.1 Update CityInfo Configuration

In `mainrunner.py`, add new location information with the following parameters:

```
'CityInfo': {
    'utm_zone_code': '',   # Get from GIS team
    'utm_zone_number': '', # Get from GIS team
    'data_storage_type': '',# From shared folder configuration
    'storage_root': '',    # From shared folder path
    'filename_preamble': '' # From shared folder naming convention
}
```

Required Actions:

1. Contact GIS team for:
    - `utm_zone_code`
    - `utm_zone_number`
2. Check shared folder structure for:
    - `data_storage_type`
    - `storage_root`
    - `filename_preamble`

2.2 Update Parameters

In `params.py`, add location-specific details:

```
default_input = {
    "sanctionLoad": 20000,  # Default value - adjustable based on researcher
requirements
    "typeOfComputation": "op",  # Options:
                                # "op" - optimized (for all HDFs)
                                # "cu" - custom (for specific polygons)
    "polygonDetails": [],  # Leave empty for all polygons
                           # Add specific details if using "cu" computation
```

```
    "city": "NEW_LOCATION_NAME"  # Add your new location name here
}
```

Configuration Notes:

- `sanctionLoad`: Default is 20000, modify if requested by researchers
- `typeOfComputation`:
  - Use "op" for processing all HDFs
  - Use "cu" for specific polygon processing
- `polygonDetails`: Required only for "cu" computation type
- `city`: Enter the new location name

3. Database Setup

3.1 Create PostgreSQL Database

```
-- Connect to PostgreSQL as admin
CREATE DATABASE [state_location_name]_[location_name];
-- Example: rtse_kt_mysore
```

3.2 Database Naming Convention

- Prefix: location state name
- Suffix: location name in lowercase
- Example: `karnataka_mysore` for Mysore location

4. Running the Process

4.1 Preferred Execution Environment

- Run on the server for full location data processing
- Command: `python mainrunner.py`

4.2 Execution Steps

1. Verify all configuration parameters
2. Ensure database is created and accessible
3. Run on server environment
4. Monitor logs for process completion

5. Verification Checklist

Before Running:

- [ ] UTM zone information confirmed with GIS team
- [ ] Shared folder paths verified
- [ ] Parameters updated in params.py
- [ ] Database created with correct naming convention
- [ ] Server access confirmed

After Running:

- [ ] Check database for successful data dump
- [ ] Verify log files for any errors
- [ ] Confirm data processing completion
- [ ] Validate output data structure
- [ ] run report SQL script

6. Troubleshooting Common Issues

1. UTM Zone Errors:
   - Double-check values with GIS team
   - Verify format matches existing locations
2. Storage Access Issues:
   - Confirm shared folder permissions
   - Verify path structure
3. Database Connection Issues:
   - Check PostgreSQL service status
   - Verify database name and permissions
4. Processing Errors:
   - Check log files for specific error messages
   - Verify input data format matches expectations

7. Key Contacts

- GIS Team: For UTM zone information
- System Admin: For server access and shared folder permissions